

Final Merge-Readiness Architecture Review

Fe origin-overhaul-phased + Sonatina trace-debug-content

Prepared for PR presentation and final architecture review

Date: 2026-05-24

Scope note: This is a static source review of the public branches and linked files. I did not independently clone, run cargo, run the CLI smoke matrix, or verify CI wall-clock times in this sandbox. Treat build/test/performance conclusions as architecture and code-shape guidance that should be confirmed by CI before merge.

1. Executive verdict

Verdict: the two branches are in the right architecture basin and are plausible PR candidates, but the Fe branch should not be presented as merge-ready until a short P0 polish/correctness list is resolved. The Sonatina branch is much closer to merge-ready as a substrate PR, assuming its claimed inliner propagation tests and formatting/check suite pass.

- Fe branch: strong architecture, but gate merge on report/documentation honesty, gas duplication, DebugBundle extent/source attribution SSOT issues, and reachability/index performance before enabling LSP/background introspection broadly.
- Sonatina branch: directionally merge-ready as a frontend-neutral trace/debug/content-addressing substrate. It has the right non-goals: no Fe identity, no Cranelift dependency, no EVM/SP1 specifics. Add hash-schema/version notes and propagation golden tests if not already present.
- Cross-branch story: good. Sonatina owns frontend-neutral SSA/CFG trace views and opaque debug handles; Fe adapts its canonical OriginExportKey/TraceFact model into those handles and consumes Sonatina views to emit validated trace bundles.

Recommended PR stance: Open both as PRs once the Fe P0 items are patched. If those items are not patched, open Fe as a draft architecture PR and Sonatina as the substrate PR. The PR description must clearly separate what is compiler-emitted today from what is inferred, posthoc-classified, or still planned.

2. Merge-readiness matrix

Area	Status	Review finding
Typed phase-owned identity	Pass	OriginKey/OriginExportKey remain the core spine; HIR/MIR/codegen/Sonatina ownership is correctly separated.
SSOT authority zones	Mostly pass	Facts/debug/query/export are derived views. Remaining risks: DebugBundle recomputes extents; gas has overlapping records; relation schemas are manual trait implementations.
Trace bundle validity	Mostly pass	Trace-facts schema is broad and typed. Validator has a good baseline; needs stronger cross-field and duplicate checks before public-ish JSONL stability.
Report honesty	Needs polish	README/status text and branch claims must align. Zext causality, backend storage, and target-loop membership should be labeled emitted/inferred/missing per fact availability.
Content addressing	Mostly pass	BLAKE3/SCC/dimensions are strong. FNV fingerprints still appear in runtime CFG/trace contexts and must be renamed as fingerprints or replaced where used as content addresses.
Datalog base emission	Pass for MVP	TraceRelation trait correctly derives base relations from typed facts; only

		core reachability rules exist. Avoid re-growing a handwritten relation system.
DWARF/ethdebug	Pass as MVP exports	Line-table and instruction/source MVPs are appropriately narrow and fail closed on low confidence. Do not present as full variable-location debug yet.
LSP/CLI integration	Promising, gated	Shared IntrospectionService and .fe-lsp.json live connection are good; live recomputation needs debounce/caching/limits before background defaults.
Sonatina frontend neutrality	Pass	Opaque handles and trace-view APIs do not leak Fe types. Good base for future frontends and Cranelift.
Performance readiness	Needs gate	Opt-in gating is good. Reachability and LSP recomputation need caching/memoization before large-project use.

3. Required gates before merge

3.1 P0 blockers or merge gates

ID	Gate	Why it matters
P0-1	Reconcile README/status/claims	The visible README and trace CLI status still distinguish fixture-backed demos and name loop membership/zext causality gaps. If the latest code now closes these gaps, update the docs/status/output. If not, keep the PR claim conservative. This is a presentation and trust blocker.
P0-2	Eliminate gas double-counting	Codegen emits both GasCostFact and StaticGasFact for the same instruction; DebugBundle currently appears to ingest both. Choose one canonical static gas record or make one a derived/legacy view, and make gas reports prove no double counting.
P0-3	Use InstructionExtentFact as authoritative PC/range	DebugBundle should not parse bytecode PC ranges from origin local keys when InstructionExtentFact exists. This is an SSOT issue and can corrupt debug/export ranges.
P0-4	Fix source attribution classification for unmapped/synthetic edges	Do not classify OriginEdgeLabel::Unmapped as synthetic. Distinguish SourceMapped, Synthetic, BackendPrepared, Ambiguous, and Unmapped explicitly with reasons.
P0-5	Cache or memoize reachability for LSP/live use	source_candidates_for_instruction / origin_reaches-style transitive search should not recompute a full traversal per instruction on large traces. At minimum cache per snapshot in TraceIndex.
P0-6	FNV naming/usage discipline	FNV64 is still visible as cfg_hash/trace fingerprint in some paths. If it is not cryptographic, call it fingerprint and exclude it from content-address/security claims, or switch those exported fields

3.2 P1 fixes that should land before or immediately after PR review

- Strengthen TraceValidator: duplicate instruction offsets per code object, duplicate facts, missing origin nodes for function/code object/source, invalid or empty free-form strings, loop membership cross-function consistency, and instruction extent validity.
- Add golden JSON output tests for CLI and LSP agent output, including confidence headers and proof that reports state whether facts are emitted, inferred, posthoc, or unavailable.
- Add a relation coverage test: every TraceFact variant maps to exactly one intended base relation or is explicitly non-relational.
- Document hash schema versions for Fe shape-address and Sonatina content_address. Hashes should be expected to churn when schema versions change.
- Add Sonatina propagation golden tests for preserved, copied, split, merged, synthetic, and unknown debug-handle cases if they are not already present.
- Add a small large-trace synthetic benchmark or unit test for reachability and DebugBundle construction to prevent $O(N*M)$ regressions.
- Ensure .fe-lsp.json token and endpoint diagnostics are explicit and no background introspection runs unless enabled by config or command-line flag.
- Label DWARF DW_LANG_C fallback and ethdebug MVP schema as compatibility placeholders, not complete official debug-format support.

4. Fe branch architecture review

4.1 Identity and authority zones

Finding: The identity spine is substantially correct. common remains mostly boring: OriginKey and OriginExportKey provide mechanics; phase crates own the meaning of phase-local identities; trace-facts/debug/query/export are downstream.

- OriginKey keeps owner and local identity together. This prevents body-local, block-local, or instruction-local IDs from becoming accidental global identifiers.
- OriginExportKey separates kind, owner_key, and local_key, with validated constructors and serde validation. This is the right external key shape.
- MIR now owns MIR/runtime local, block, statement, terminator, CFG, loop, storage, and value-property facts. This is the correct owner boundary.
- Codegen owns EVM bytecode PC/opcode/category/static gas/code-object facts. This is the correct target-specific boundary.
- DebugBundle, DWARF, ethdebug, Datalog, CLI, and LSP are downstream consumers. They must stay derived.

Remaining concern: trace-facts is necessarily broad now. That is acceptable because it serializes compiler-emitted facts, but it must not become a semantic registry. Every entity field should remain an OriginExportKey; names/spans stay display metadata.

4.2 Trace facts, JSONL, and relation emission

The TraceFact model now covers origin nodes/edges, compiler events, storage, instructions, categories, blocks, CFG edges, loops, instruction extents, inline contexts, opcodes, gas, source files/spans, code objects, functions, scopes, types, variables, location ranges, dynamic gas, and shape hash records. That breadth is appropriate for the target architecture.

- The relation layer is currently much healthier than the old relation-table spike because TraceRelation is attached to typed facts. Keep it that way.
- Do not reintroduce independent column-order registries, separate validators, or report-specific relation schemas.
- The current manual trait implementation is acceptable for this PR if covered by relation coverage tests. A derive macro can come later.

4.3 MIR facts and loop facts

MIR trace emission is a real architectural improvement. It emits runtime functions, locals, blocks, statements, terminators, CFG edges, natural loops, loop blocks, storage facts, value properties, compiler events, and loop shape hash facts from the MIR crate itself. That is the right owner.

- Good: MIR explicitly does not claim backend stack slots or target-specific storage. It classifies MIR-level storage/location and leaves backend allocation to codegen/backends.

- Good: loop detection over MIR CFG gives a phase-owned loop story rather than relying only on final-code pattern matching.
- Polish: runtime_cfg_hash appears to be FNV64. If exported or used as content address, switch to BLAKE3 or rename to runtime_cfg_fingerprint.
- Polish: if MIR and codegen duplicate dominator/natural-loop helpers, consider a shared analysis helper later. Do not block merge on this unless bugs appear.

4.4 Codegen and EVM facts

Codegen is now emitting real target-layer facts from emitted EVM bytecode: bytecode PC identity, instruction extents, opcode facts, instruction categories, static gas facts, code objects, and content hashes. This is a major improvement over fixture-only demonstrations.

- Good: codegen owns EVM opcode/gas/bytecode extent facts. That is the correct target-specific authority zone.
- Good: categories are labeled with a category source, so backend-emitted vs posthoc classifier status can be represented.
- Concern: GasCostFact and StaticGasFact overlap. If both are emitted and consumed, gas reports can double-count. Pick one canonical static gas representation.
- Concern: code object hashes should be cryptographic if they are treated as content addresses. If they are fast fingerprints, label them accordingly.

4.5 Sonatina bridge inside Fe

The Fe branch now contains a Sonatina trace-view adapter that can consume SonatinaTraceView and emit Sonatina function, block, instruction, CFG, instruction-block, and loop facts, plus edges from Sonatina instructions back to frontend-origin handles when present. This is the right consumption model for the Sonatina substrate branch.

- Good: Fe adapts Fe OriginExportKey into frontend-origin records rather than forcing Fe types into Sonatina.
- Good: the deprecated MIR-derived Sonatina bridge is labeled transitional/test-only. Keep that label loud until it is deleted.
- Merge gate: reports must distinguish loops derived from MIR CFG, Sonatina CFG, target bytes, or fixtures. Do not conflate them.

4.6 DebugBundle, DWARF, and ethdebug

The DebugBundle builder is the correct architectural idea: create a derived debug/export model from validated trace facts, then feed DWARF, ethdebug, LSP, and CLI views from that model. The implementation still has two correctness issues that should be fixed before merge.

Type	Finding
Good	DWARF line table emits only high-confidence source-mapped rows and fails closed when no source rows exist.
Good	ethdebug MVP emits source/material/program/instruction records with source confidence and Fe origin-key sidecar information.
Fix	DebugBundle should use InstructionExtentFact for PC ranges instead of reconstructing ranges from origin local strings.
Fix	DebugBundle source candidate traversal should be memoized and label-aware; it should not treat all edges as equally valid attribution paths.
Fix	Unmapped/backend-prepared/synthetic classifications should remain separate in debug exports and reports.

4.7 CLI/LSP and IntrospectionService

The shared IntrospectionService and live CLI-to-LSP flow are exactly the right direction. The server-info and .fe-lsp.json contract gives the CLI/agent a live handle into the LSP without inventing a second introspection path.

- Good: LSP features are config-gated and route through the same trace/introspection service.
- Good: the live client uses a versioned server-info file, token, endpoint, root, server PID, feature set, and config hash.
- Risk: the visible LSP service path builds MIR trace facts and codegen bytecode facts per request. This is not safe for background-by-default behavior on large projects.
- Recommendation: keep live/background introspection off by default, add debounce/cancellation/max-facts/max-ms controls, and move heavy trace construction into salsa-tracked or cached queries before enabling on-save background mode.

5. Sonatina branch architecture review

5.1 Frontend neutrality

Finding: The Sonatina branch appears to meet the frontend-neutrality goal. It introduces FrontendOriginId, DebugLocId, DebugTagId, FrontendOriginRecord, DebugMetadata, and SonatinaTraceView without leaking Fe OriginExportKey, Fe HIR/MIR types, EVM gas, Cranelift, Wasmtime, or SP1 concepts into public APIs.

- Good: external_key is opaque to Sonatina. Sonatina stores and propagates handles; frontends own meaning and validation.
- Good: source spans and display labels are metadata, not identity. This is compatible with Fe, other frontends, MLIR-like adapters, or future DSLs.
- Good: SonatinaTraceView exposes function/block/instruction/CFG/debug views without requiring backend-specific knowledge.
- Risk: ExternalDebugKey/String is intentionally opaque, so each frontend adapter must validate its own encoding. Fe should only serialize canonical OriginExportKey values into this field.

5.2 Debug-handle propagation

The debug-handle design is conceptually right: Sonatina should preserve/copy/combine opaque origin/debug handles through IR construction and transformations, while classifying transformations conservatively. The PR should have tests that prove the propagation rules, because this is where overclaiming could sneak in.

- Preserved instruction: debug loc and tags survive unchanged.
- One-to-one replacement: copied debug handle is marked as copied/replacement, not falsely preserved.
- Split instruction: child instructions carry split-from classification or explicit shared origin.
- Merge: merged instructions carry multi-origin/merged classification, not a single precise source.
- Synthetic insertion: instruction has SyntheticReason or Unknown, never a fake source loc.
- Unknown transform: classification stays Unknown rather than SourceMapped.

5.3 SonatinaTraceView API readiness

API area	Readiness	Comment
Functions/blocks/instructions	Ready for v1	Enough to emit target-neutral trace nodes and loop contents.
CFG edges	Ready for v1	Block successors enable loop derivation downstream.
Instruction kinds	Ready for v1	Enough for opcode/kind-sensitive shape hashes and rough category views.
Values/def-use	Future	Needed for strong local/taint/value-flow analysis; not required for the first PR.
Loop analysis	Future/derived	Prefer deriving from CFG facts in Fe/trace-query first. Sonatina can expose loop analysis later if useful.
Target emission edges	Future target adapters	EVM/Cranelift/native projections should be implemented outside Sonatina core.

5.4 Sonatina content addressing

The content-addressing design is strong: BLAKE3, explicit dimensions, identity-bound vs elided modes, BTreeMap/BTreeSet deterministic ordering, and SCC condensation for cyclic CFGs. This is the right level for a Sonatina substrate PR.

- Good: dimensions separate structure, opcode/kind, type-sensitive, constants, debug-origin-sensitive, debug-origin-elided, layout-sensitive, and layout-elided views.
- Good: SCC condensation gives a deterministic way to hash cyclic CFG regions without pretending the graph is a tree.
- Caveat: graph hashing over SCCs is not a mathematical graph-isomorphism proof. It is a deterministic content-addressed compiler view. Document that.
- Caveat: if hash inputs use Debug formatting or textual IR, hash stability is only promised within the schema/version. Add a hash schema version string and tests that expected dimensions change independently.

6. Cross-branch integration review

Fe frontend/MIR meaning

- > Fe OriginExportKey / TraceFact
- > Sonatina frontend-neutral debug handles
- > SonatinaTraceView over SSA/CFG
- > Fe codegen target facts: EVM bytecode/opcode/gas/extent
- > DebugBundle / Datalog / DWARF / ethdebug / CLI / LSP

- The architecture is correct if Fe treats Sonatina handles as a bridge, not as replacement identity.
- The Sonatina branch is correct if it never needs to parse Fe external keys or know Fe semantics.
- The Fe branch is correct if target facts stay target-owned and reports state confidence/availability for every attribution.
- Cranelift compatibility is preserved because v1 is on Sonatina main and target-neutral. A future Cranelift adapter can consume SonatinaTraceView without reworking the core substrate.

7. Performance and incremental compilation assessment

7.1 CI time impact

The new surface area is large: trace-facts, trace-query, shape-address, debug-export, MIR trace tests, codegen trace tests, CLI smoke tests, LSP integration tests, DWARF, ethdebug, gas attribution, and command matrix tests. This can become expensive if every path runs on every push.

Tier	Recommended contents
Fast path, every PR push	cargo check workspace; targeted unit tests for common/origin, trace-facts validation, shape-address, mir trace, codegen trace, Sonatina substrate unit tests.
Medium path, required before merge	CLI smoke matrix on small fixtures; JSON golden outputs; LSP server-info smoke; debug-export DWARF/ethdebug validation; relation export tests.
Slow/nightly or merge queue	Full command matrix across optimization levels, gas dynamic traces, large fixtures, fuzz seeds, browser/LSP live integration, sanitizer-like validation.

- Use cargo nextest if available; cache target directories in CI; keep external tool checks feature-gated or smoke-sized.
- DWARF/ethdebug tests should validate schema/line-table shape on tiny fixtures, not compile a large corpus on every push.
- The full matrix should be a release/merge-queue confidence test, not an edit-loop tax.

7.2 TraceIndex reachability and source_candidates performance

Finding: The current reachability/source-candidate path appears to recompute graph traversal per instruction. This is fine for tiny traces, but it will become $O(\text{instructions} * \text{graph_size})$ on real contracts and unacceptable for LSP hovers/inlays.

Recommended immediate design:

- Build outgoing and incoming adjacency once per TraceSnapshot.
- Cache source_candidates_for_instruction by instruction OriginExportKey.
- Cache origin_reaches(start) lazily with a visited set and cycle handling.
- For large traces, consider compact integer node IDs plus bitsets or roaring bitmaps for hot reachability queries.
- Store the cache under a TraceSnapshot hash/fingerprint so repeated CLI/LSP queries share the index.
- Add a synthetic test with many instructions sharing the same origin tail to prove traversal is amortized.

```
struct TraceIndex {
    snapshot_id: TraceSnapshotId,
    nodes: Vec<OriginExportKey>,
    outgoing: Vec<Vec<EdgeId>>,
    incoming: Vec<Vec<EdgeId>>,
    source_candidates_cache: HashMap<NodeId, Vec<SourceCandidate>>,
    reaches_cache: HashMap<NodeId, BitSet>,
}
```


7.3 Salsa-tracked queries

Do not make trace emission a hidden side effect of compilation queries. But for LSP and large projects, key trace artifacts should become explicit salsa-tracked queries once the API stabilizes.

Classification	Path	Reason
Good salsa query candidate	runtime_mir_trace_facts(module/function)	Pure, phase-owned, reused by CLI and LSP.
Good salsa query candidate	sonatina_trace_view_facts(function, opt_level, policy)	Depends on Sonatina substrate; avoids recomputing loop/CFG facts.
Good salsa query candidate	bytecode_instruction_facts(code_object, target, opt_level)	Target-specific, stable for unchanged codegen object.
Good salsa query candidate	shape_hashes(entity, policy)	Potentially expensive; cache by policy/entity.
Avoid as query output	serialized JSONL strings	Keep typed facts cached; serialize only at command/export boundary.
Avoid	global mutable trace sink side effects	Breaks incremental determinism and makes tests flaky.

7.4 Opt-in gating and LSP performance

The current opt-in story is the right starting point: explicit trace commands, trace mode configuration, and LSP feature gates. It is sufficient to prevent normal compilation regressions if defaults stay conservative.

- Default normal compilation path should not emit trace JSONL, shape hashes, debug exports, Datalog relations, or live LSP introspection unless requested.
- LSP on-save should default to lightweight diagnostics only; trace inlays/hovers/code lenses should require config flags and debounce.
- Background trace mode should have max file size, max fact count, max time budget, cancellation, and stale-result handling.
- CLI should print whether expensive features were enabled: shape hashes, debug bundle, Datalog export, DWARF, ethdebug, dynamic gas attribution.
- Agent JSON output should include trace size, build duration, query duration, and whether results came from cache.

8. Content-addressing discipline

Bottom line: The cryptographic content-addressing story is strong only if all artifact-address claims use cryptographic digests and all fast fingerprints are labeled as such. Do not let FNV64 appear in a report that sounds like artifact attestation.

Kind	Definition	Use
Cryptographic content address	BLAKE3 or SHA-256 digest over canonicalized bytes/facts/shapes.	Use for artifact identity, regression witnesses, cache keys intended to be robust, attestation hooks.
Fast fingerprint	FNv64 or similar non-cryptographic hash.	Use only for internal bucketing, debug summaries, or quick change detection; label as fingerprint.
Anonymous shape hash	Elides identity/origin/name as configured.	Use for structural equivalence, fuzz dedup, frontend-neutral comparisons.
Identity-bound hash	Includes canonical OriginExportKey / Sonatina identity.	Use for precise per-compilation artifact tracking, not cross-run structural equality.
Debug-origin-sensitive hash	Includes origin/debug handles.	Use when provenance changes should invalidate comparison/cache.
Debug-origin-elided hash	Excludes origin/debug handles.	Use when equivalent generated code should compare equal despite frontend provenance differences.

- Add a hash_policy_id and hash_schema_version to all shape/content-address records.
- Length-prefix every string/byte field in hash input. Avoid ambiguous concatenation.
- Avoid unstable Debug formatting in long-term content addresses, or explicitly version the textual encoding.
- For cyclic CFGs, SCC condensation is a deterministic compiler-view policy, not proof of graph isomorphism. Use it as an engineering content-address, not a formal equivalence proof.

9. PR presentation guidance

The PR story should be ambitious but careful. The strongest honest pitch is:

These branches make Fe and Sonatina trace-ready. They do not claim a formally verified compiler. They make each compilation auditable: phase-owned identity, validated trace facts, content-addressed IR views, derived debug/query/export models, and explicit confidence labels for every report.

9.1 Claims to make

- Phase-owned typed identity prevents raw local IDs from becoming global provenance.
- Trace bundles are validated, JSONL-serializable evidence about a specific compilation.
- MIR owns MIR/runtime local/storage/CFG/loop facts; codegen owns EVM bytecode/opcode/gas facts; Sonatina owns frontend-neutral SSA/CFG trace views.
- Debug/Datalog/DWARF/ethdebug/LSP/CLI consume derived facts; they do not own compiler truth.
- Content addressing supports granular regression, fuzzing, and future artifact attestation when cryptographic digests are used.
- Reports carry confidence labels and should say when evidence is missing, inferred, ambiguous, synthetic, or unmapped.

9.2 Claims to avoid until facts prove them

- Do not claim full optimizer provenance; claim conservative pre/post snapshot and debug-handle propagation where implemented.
- Do not claim precise backend storage/spill causality unless backend storage/location facts exist.
- Do not claim zext causality unless InsertIntegerZeroExtend or equivalent compiler events exist.
- Do not claim target-loop bytecode membership unless Sonatina-to-target emission edges and loop membership facts connect the chain.
- Do not claim formal compiler verification. Claim per-compilation static evidence and validation.

10. Final acceptance checklist

Area	Merge-ready condition
Docs	README, CLI status, and PR description accurately distinguish emitted, inferred, fixture, posthoc, and unavailable facts.
Tests	cargo check/test pass; fast and medium CI tiers defined; command smoke matrix passing on tiny fixtures.
Trace validation	Validator catches duplicate/missing endpoints, bad extents, malformed strings, loop/function mismatches, and gas duplication.
Gas	One canonical static gas source; dynamic gas attribution explicitly execution-specific; no double-counting.
DebugBundle	Uses InstructionExtentFact; label-aware source attribution; cached reachability; no raw string identity.
Content addressing	BLAKE3 for content addresses; FNV only labeled as fingerprint; schema versions present.
LSP	Live introspection disabled or conservative by default; debounce/cancel/limits in place; server-info diagnostics clear.
Sonatina	No Fe leakage; debug propagation tests; hash determinism tests; trace-view docs; no Cranelift dependency.
PR narrative	Safety story is per-compilation auditable evidence, not formal verification.

Appendix A. Evidence reviewed

Representative public branch/files reviewed. Line-level citations were inspected through GitHub web views; exact line numbers may move as branches continue to change.

- Fe branch: <https://github.com/micahscopes/fe/tree/origin-overhaul-phased>
- Fe common origin: [crates/common/src/origin.rs](https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/common/src/origin.rs)
- Fe trace facts: [crates/trace-facts/src/fact.rs](https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/trace-facts/src/fact.rs), [relation.rs](https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/trace-facts/src/relation.rs), [validate.rs](https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/trace-facts/src/validate.rs), [evm_trace.rs](https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/trace-facts/src/evm_trace.rs)
- Fe MIR trace: [crates/mir/src/trace.rs](https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/mir/src/trace.rs)
- Fe codegen trace: [crates/codegen/src/trace.rs](https://github.com/micahscopes/fe/blob/origin-overhaul-phased/crates/codegen/src/trace.rs)

- Fe shape-address: crates/shape-address/src/lib.rs
- Fe trace CLI/live: crates/fe/src/trace/trace_cli.rs, trace_live.rs
- Fe LSP introspection: crates/language-server/src/introspection.rs
- Fe debug export: crates/debug-export/src/model.rs, dwarf.rs, ethdebug.rs
- Fe Datalog relation export: crates/trace-query/src/datalog_emit.rs
- Sonatina branch: <https://github.com/micahscopes/sonatina/tree/trace-debug-content>
- Sonatina scope document: docs/trace-debug-content-addressing.md
- Sonatina debug handles: crates/ir/src/debug.rs
- Sonatina trace view: crates/ir/src/trace_view.rs
- Sonatina content address: crates/ir/src/content_address.rs

Appendix B. Suggested concrete patches

B.1 DebugBundle extent and attribution patch

1. Add InstructionExtentFact indexing to DebugFactIndex.
2. Replace instruction_pc_range(local_key parsing) with:
`extent_for(instruction).or_else(legacy_parse_pc_range)`
3. Make source_candidates label-aware:
`allowed direct source labels: LoweredFrom, EmittedFrom, SyntheticFor (confidence adjusted)`
`explicit Unmapped stops source attribution`
`BackendPrepared yields Synthetic/BackendPrepared, not SourceMapped`
4. Cache source_candidates per instruction.
5. Add tests:
 - instruction extent wins over local-key parsing
 - Unmapped edge stays Unmapped
 - synthetic edge is Synthetic with reason
 - repeated source candidate query hits cache / does not re-walk graph

B.2 Gas canonicalization patch

1. Decide canonical static gas fact:
`Preferred: StaticGasFact for schedule/base/dynamic_kind/confidence.`
2. Treat GasCostFact as deprecated view or delete it from codegen emission.
3. DebugBundle::gas should not push both records for one instruction.
4. Add validator warning/error for duplicate static gas facts per instruction/schedule/policy.
5. Add JSON golden test for gas-by-source that proves no double-counting.

B.3 FNV vs BLAKE3 patch

1. Audit exported fields containing "hash" and "fnv64".
2. Rename non-cryptographic values to *_fingerprint.
3. Use BLAKE3 for content addresses and public trace/debug bundle hashes.
4. Add tests that content-address fields use blake3: or a 64-hex digest, not fnv64:.
5. Keep FNV only for internal maps or explicitly labeled quick fingerprints.

B.4 LSP performance guard patch

1. Default trace inlays/hovers/code lenses off unless config enables them.
2. Add debounce and cancellation for trace requests.
3. Add max_trace_facts, max_shape_nodes, max_query_ms.
4. Cache TraceSnapshot and TraceIndex per document version/config hash.
5. Include cache_hit and query_duration_ms in agent JSON output.